



Sara Jadhav (24060622010)

Data Science - I

Guided By - Prof. Monika Vishnoi

EDA PROJECT



Uncovering Customer Behavior and Sales Patterns:
Exploratory Data Analysis (EDA) of an Online Retail Dataset





Project Agenda



- Project Objective
- Data Sources & Citation
- Database Schema Overview
- Methodology and Tools
- Data Cleaning & Preprocessing
- Exploratory Data Analysis (EDA)
- SQL Analysis & Key Queries
- Limitations & Challenges
- Lessons Learned
- Key Takeaways
- Future Recommendations
- Conclusion

Project Objective

SCOPE

UK-based online retailer, real world transactional data

541,909 transactions (Dec 2010 - Dec 2011)

38 countries, 4338 customers, 3922 products

8 columns → 15+ engineered features

Aim:

Perform a comprehensive Exploratory Data Analysis on the Online Retail Dataset to uncover customer behavior, sales patterns, and business insights.

Specific Objectives:

- Understand data quality and clean the dataset.
- Identify key sales trends (time, product, geography).
- Segment customers using RFM and cohort analysis.
- Derive actionable recommendations for business growth.

Expected Outcomes:

- Understand customer purchasing habits
- Identify high-value customers & segments
- Analyze sales trends (time, geography, product)
- Provide actionable recommendations



Data Source & Citation

Dataset: Online Retail Dataset

Source: Kaggle

Owner: Vijaykumar Ummadisetty

Citation:

[https://www.kaggle.com/datasets/vijayuv/onlineretail?
resource=download](https://www.kaggle.com/datasets/vijayuv/onlineretail?resource=download)

RAW ONLINE RETAIL DATA SNAPSHOT



InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/01/10 8:26	2.55	17850	United Kingdom
536365	71053	WHITE METAL LANTERN	6	12/01/10 8:26	3.39	17850	United Kingdom
536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/01/10 8:26	2.75	17850	United Kingdom
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/01/10 8:26	3.39	17850	United Kingdom
536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/01/10 8:26	3.39	17850	United Kingdom

Database Schema Overview

After Cleaning & Feature Engineering:

Two datasets created:

- **retail_full (519,589 rows)** – all cleaned transactions, keeps missing Customer IDs for sales analysis.
- **retail_customers (387,844 rows)** – only rows with valid Customer ID, used for customer-centric analysis.

Derived Features:

- **Revenue** = Quantity × UnitPrice
- **Temporal:** Year, Month, Day, Hour, DayOfWeek
- **Cohort:** InvoiceMonth, CohortMonth, CohortIndex

Column Name	Description	Data Type
InvoiceNo	A 6-digit integral number uniquely assigned to each transaction. (Note: 'C' prefix denotes cancellations)	Object
StockCode	A 5-digit integral number uniquely assigned to each distinct product and some of them end with an alphabet.	Object
Description	The product name.	Object
Quantity	The number of units of a product purchased in a transaction.	Integer
InvoiceDate	The date and time of the transaction.	DateTime
Unit Price	The price per unit of the product in Pound Sterling (£)	Float
CustomerID	A 5-digit integral number uniquely assigned to each customer. (Significant missing values causing the data type to be recognised as Float which is fixed later)	Float
Country	The name of the country where the customer resides.	Object

KEY FEATURES OF RAW ONLINE RETAIL DATA

Methodology and Tools

Phase 1 – Data Preparation (Python – Google Colab)

- Load data, inspect, clean, handle missing values, remove duplicates, filter invalid entries
- Feature engineering (Revenue, time features)
- Split into two datasets (full and customers) for different analyses

Phase 2 – Exploratory Analysis (Python)

- Univariate, bivariate, time series, product, country, customer analysis
- RFM, cohort, CLV analysis
- Visualization: matplotlib, seaborn

Phase 3 – SQL Validation (PostgreSQL / pgAdmin4)

- Cross-check Python findings
- Advanced window functions, CTEs, retention calculation
- Reusable business queries

Data Cleaning & Preprocessing

Initial Data Issues:

- Missing Customer ID: 135,080 records (24.9%)
- Missing Description: 1,454 records (0.27%)
- Duplicates: 5,267 exact duplicate rows
- Cancelled transactions: Invoices starting with 'C'
- Negative Quantity / Unit Price: invalid entries

Actions Taken:

1. Removed cancelled transactions
2. Removed negative Quantity and Unit Price
3. Dropped duplicate rows (full duplicates)
4. Removed rows with missing Description
5. Converted CustomerID to string for consistency

Result:

- Full dataset: 519,589 records
- Customer dataset: 387,844 records

Cleaning Dataset

```
# Checking duplicates
df.duplicated().sum()

# Removing exact duplicate rows
df = df.drop_duplicates()

# Removing only true duplicates
df = df.drop_duplicates(subset=[
    'Invoice No', 'Stock Code', 'Invoice Date', 'Customer ID'
])

# Removing cancelled transactions
df['Invoice No'].astype(str).str.startswith('C').sum()

# Removing negative or zero quantity
df = df[df['Quantity'] > 0]

# Removing negative or zero price
df = df[df['Unit Price'] > 0]

# Removing missing values for Description
df = df.dropna(subset=['Description'])
```



Creating Two Datasets

```
# Full dataset (KEEP missing Customer ID for Sales Analysis)
df_full = df.copy()

# Customer dataset (REMOVE missing Customer ID for Customer Analysis)
df_customers = df.dropna(subset=['Customer ID']).copy()
```

Fixing Data Types

```
# Converting after cleaning
df_full['Customer ID'] = df_full['Customer ID'].astype('Int64').astype(str)
df_customers['Customer ID'] = df_customers['Customer ID'].astype('Int64').astype(str)
```

Featuring Engineering

```
# Creating Revenue
df_full['Revenue'] = df_full['Quantity'] * df_full['Unit Price']
df_customers['Revenue'] = df_customers['Quantity'] * df_customers['Unit Price']

# Time features
for df in [df_full, df_customers]:
    df['_Year'] = df['_Invoice Date'].dt.year
    df['_Month'] = df['_Invoice Date'].dt.to_period('M')
    df['_Month_Name'] = df['_Invoice Date'].dt.month_name()
    df['_Day'] = df['_Invoice Date'].dt.day
    df['_Hour'] = df['_Invoice Date'].dt.hour
    df['_DayOfWeek'] = df['_Invoice Date'].dt.day_name()
```


Basic Overview and Summary Statistics

Full Dataset Summary

Metric	Value
Total Orders	19,959
Average Order Value (AOV)	£530.44
Total Quantity Sold	5,546,556
Average Quantity per Order	277.90
Total Revenue	£10,587,081.95

Customer Dataset Summary

Metric	Value
Total Orders	18,531
Average Order Value (AOV)	£477.28
Total Quantity Sold	5,128,601
Average Quantity per Order	276.76
Total Revenue	£8,844,544.40

Full Dataset Summary

Total Orders (Full Data): 19959
Average Order Value (Full Data): 530.44
Total Quantity Sold (Full Data): 5546556
Average Quantity per Order (Full Data): 277.90

Customer Dataset Summary

Total Orders (Customer Data): 18531
Average Order Value (Customer Data): 477.28
Total Quantity Sold (Customer Data): 5128601
Average Quantity per Order (Customer Data): 276.76

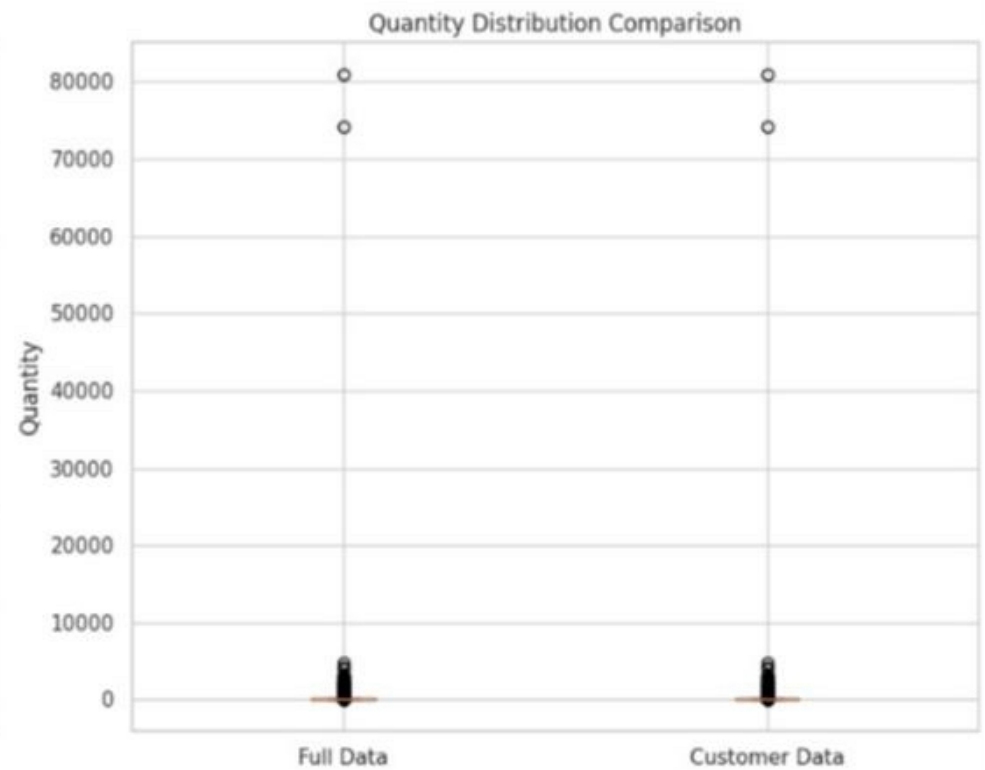
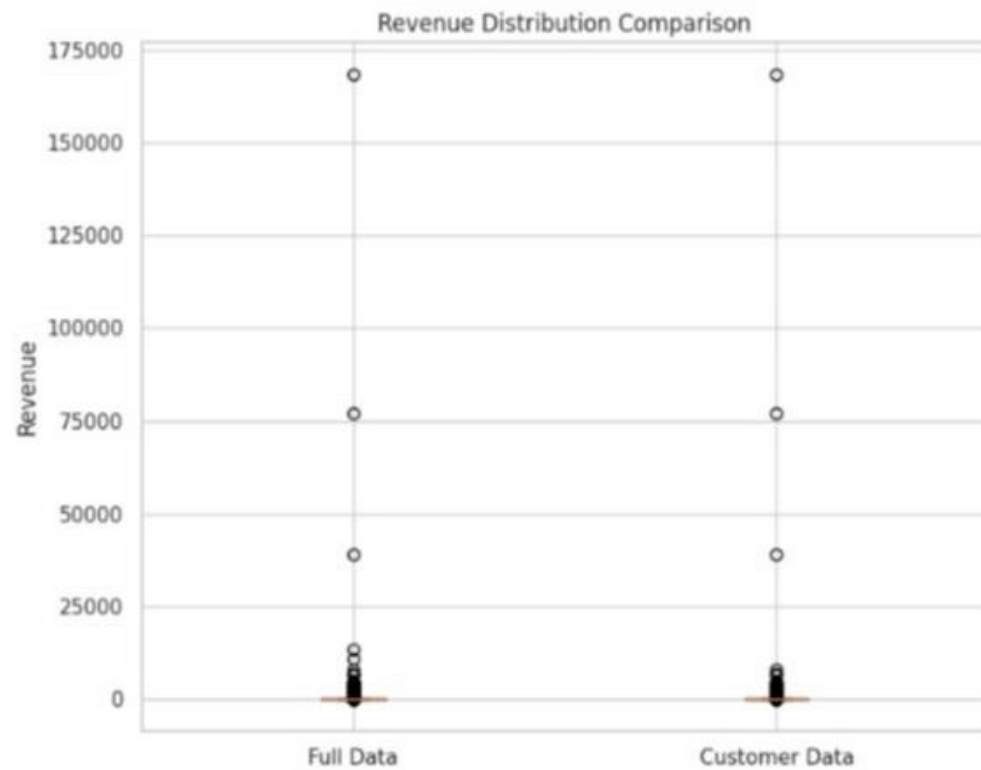
Online Retail Dataset Summary

	Metric	Full Data	Customer Data
0	Total Orders	1.995900e+04	1.853100e+04
1	Average Order Value	5.304415e+02	4.772837e+02
2	Total Quantity Sold	5.546556e+06	5.128601e+06
3	Average Quantity per Order	2.778975e+02	2.767579e+02

Online Retail Dataset Comparison Table

	Metric	Full Dataset	Customer Dataset
0	Rows	5.195890e+05	387844.000
1	Transactions	1.995900e+04	18531.000
2	Unique Customers	4.339000e+03	4338.000
3	Unique Products	3.922000e+03	3665.000
4	Countries	3.800000e+01	37.000
5	Total Revenue	1.058708e+07	8844544.403

Boxplot Outliers



Univariate Analysis



Quantity Distribution

- Highly right-skewed (most purchases 1-10 units)
- Log transformation reveals underlying normal-like distribution
- Bulk purchases (up to 80,995 units) indicate wholesale customers

fig, axes = plt.subplots(1, 2, figsize=(18, 6)) # Two subplots for normal and log comparison

Normal Scale Comparison

```
axes[0].hist([df_full['Quantity'],
df_customers['Quantity']], bins=50, alpha=0.7, label=['Full Data', 'Customer Data'], color=['blue', 'orange'])
axes[0].set_title("Quantity Distribution (Normal Scale)")
axes[0].set_xlabel("Quantity")
axes[0].set_ylabel("Frequency")
axes[0].legend()
```



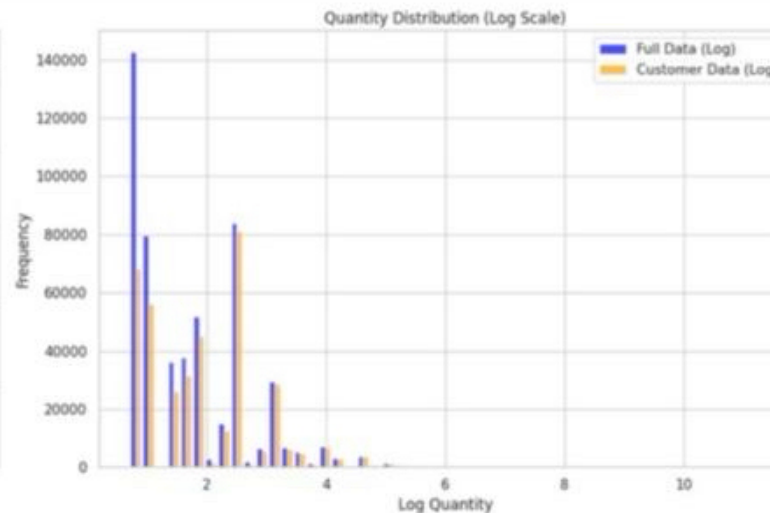
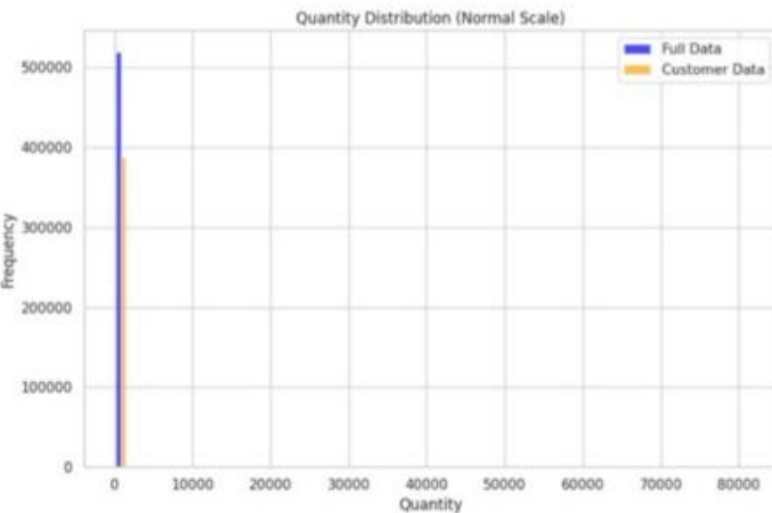
Revenue Distribution

- Similar skew - most transactions low value (£1-£50)
- Log scale shows multiple peaks → different customer segments

Log Scale Comparison

```
axes[1].hist([np.log1p(df_full['Quantity']),
np.log1p(df_customers['Quantity'])], bins=50,
alpha=0.7, label=['Full Data (Log)', 'Customer Data (L
axes[1].set_title("Quantity Distribution (Log Scale)")
axes[1].set_xlabel("Log Quantity")
axes[1].set_ylabel("Frequency")
axes[1].legend()
plt.tight_layout()
plt.show()
```


Univariate Analysis

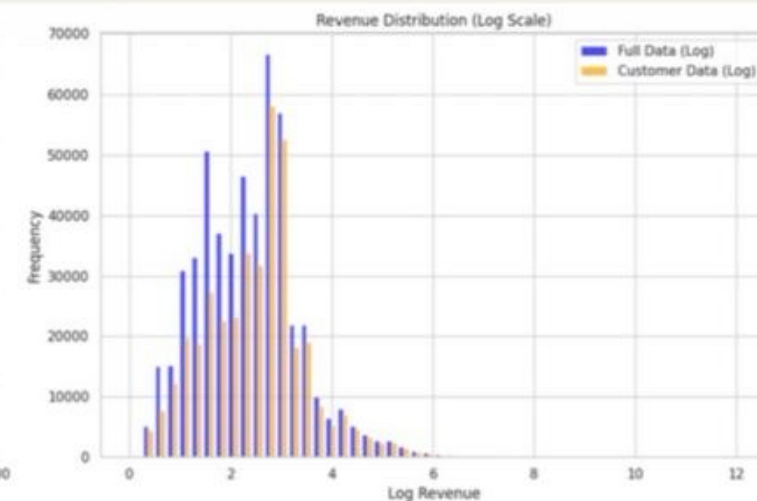
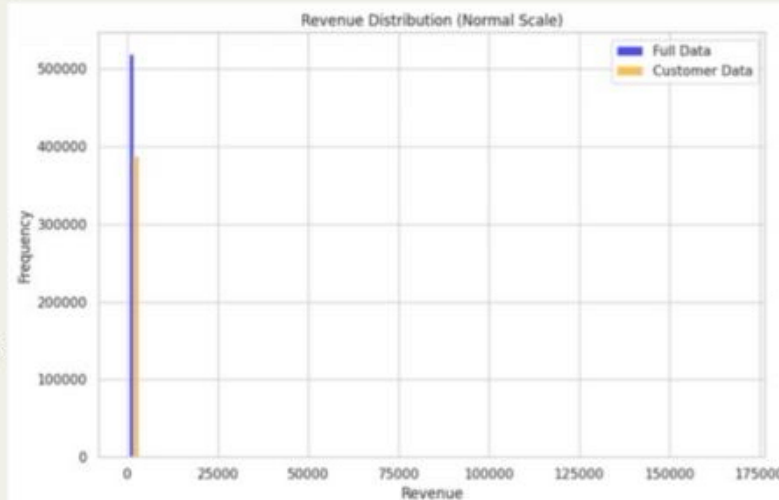


Quantity and Revenue Distribution (Normal vs Log) for both Full & Customer Datasets - Combined



Why Log Transformation?

- Normalizes skewed data
- Reduces impact of outliers
- Reveals underlying structure



Bivariate Analysis



Quantity vs Revenue

- Strong positive correlation (as expected)
- Outliers: low revenue from high quantity (low-priced items)
- Log-log plot shows linear relationship

```
fig, axes = plt.subplots(2, 2, figsize=(15,10))
```

FULL DATA - NORMAL

```
axes[0,0].scatter(df_full['Quantity'], df_full['Revenue'])
axes[0,0].set_title("Full Data - Quantity vs Revenue")
axes[0,0].set_xlabel("Quantity")
axes[0,0].set_ylabel("Revenue")
```

FULL DATA - LOG

```
axes[0,1].scatter(np.log1p(df_full['Quantity']),
np.log1p(df_full['Revenue']))
axes[0,1].set_title("Full Data - Log Quantity vs Log Revenue")
axes[0,1].set_xlabel("Log Quantity")
axes[0,1].set_ylabel("Log Revenue")
```



Quantity vs Unit Price

- Slight negative correlation: higher price → lower quantity
- Log scale makes relationship clearer
- Indicates price sensitivity for some products

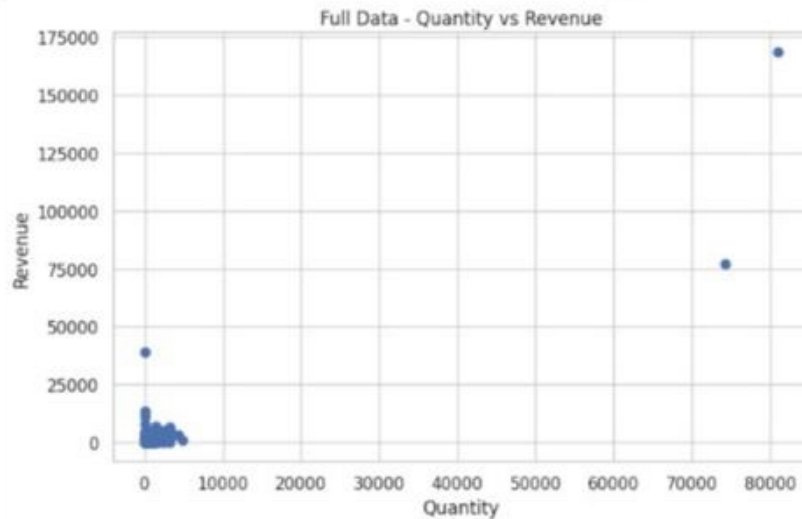
CUSTOMER DATA - NORMAL

```
axes[1,0].scatter(df_customers['Quantity'], df_customers['Revenue'])
axes[1,0].set_title("Customer Data - Quantity vs Revenue")
axes[1,0].set_xlabel("Quantity")
axes[1,0].set_ylabel("Revenue")
```

CUSTOMER DATA - LOG

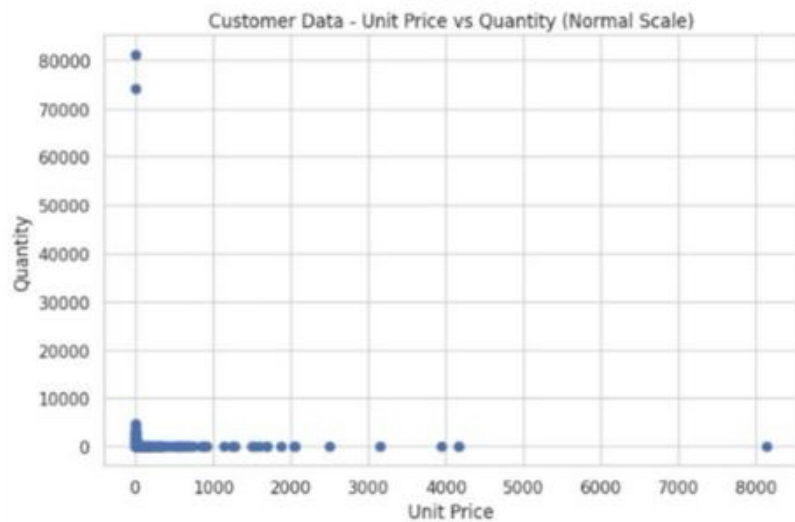
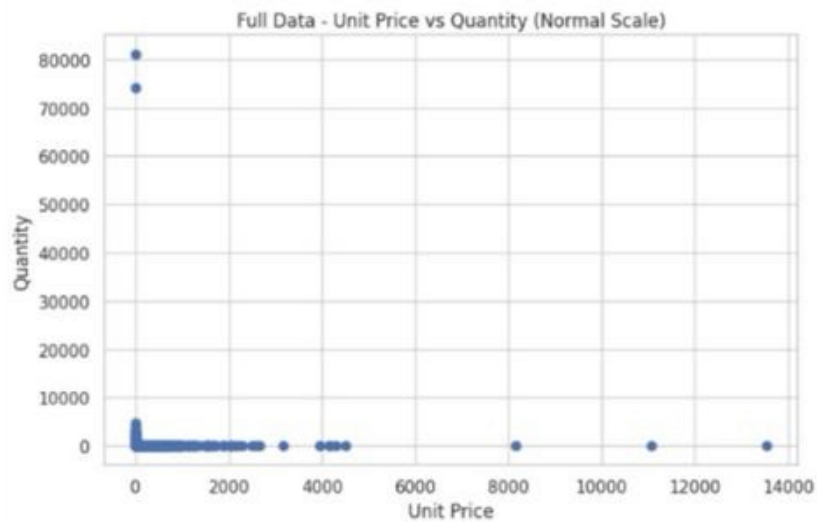
```
axes[1,1].scatter(np.log1p(df_customers['Quantity']),
np.log1p(df_customers['Revenue']))
axes[1,1].set_title("Customer Data - Log Quantity vs Log Revenue")
axes[1,1].set_xlabel("Log Quantity")
axes[1,1].set_ylabel("Log Revenue")
plt.tight_layout()
plt.show()
```

Bivariate Analysis



Scatter Plot (Normal vs Log) for Quantity vs Revenue for both Full & Customer Datasets - Combined

Bivariate Analysis



Scatter Plot (Normal vs Log) for Quantity vs Unit Price for both Full & Customer Datasets - Combined

Time Series Analysis



Monthly Revenue Trend:

- Steady growth throughout 2011
- Peak in November (holiday shopping)
- December 2010 partial month

Revenue by Day of Week:

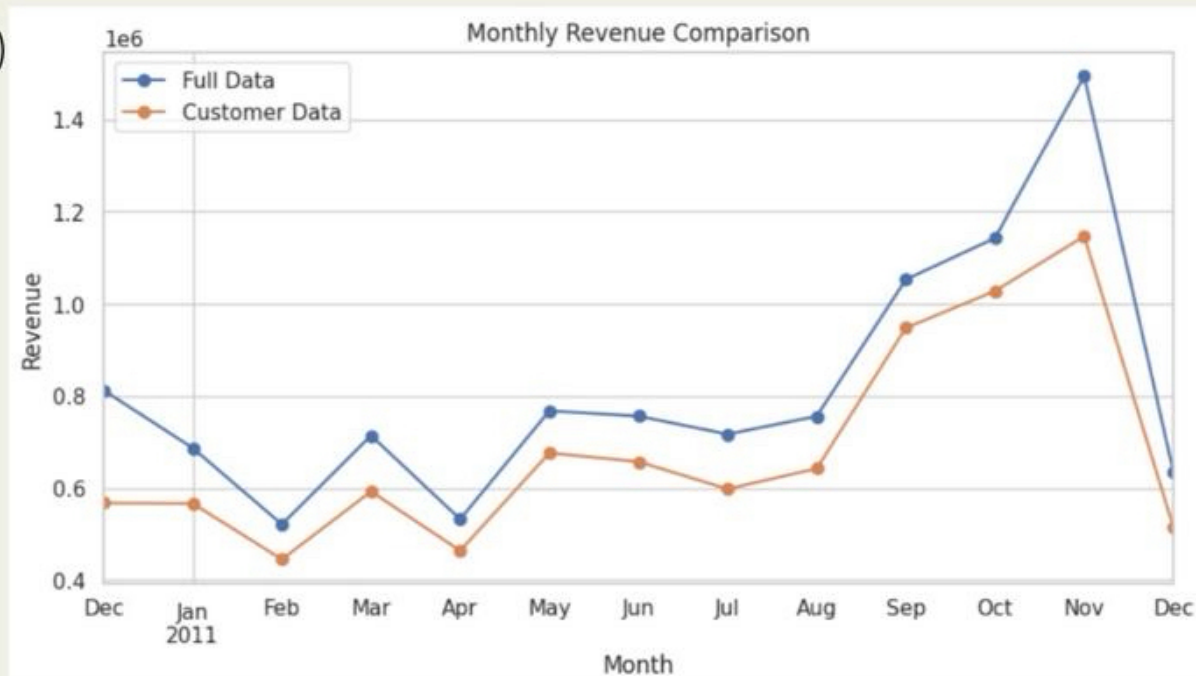
- Highest: Thursday, Tuesday, Wednesday
- Lowest: Saturday, Sunday

Revenue by Hour:

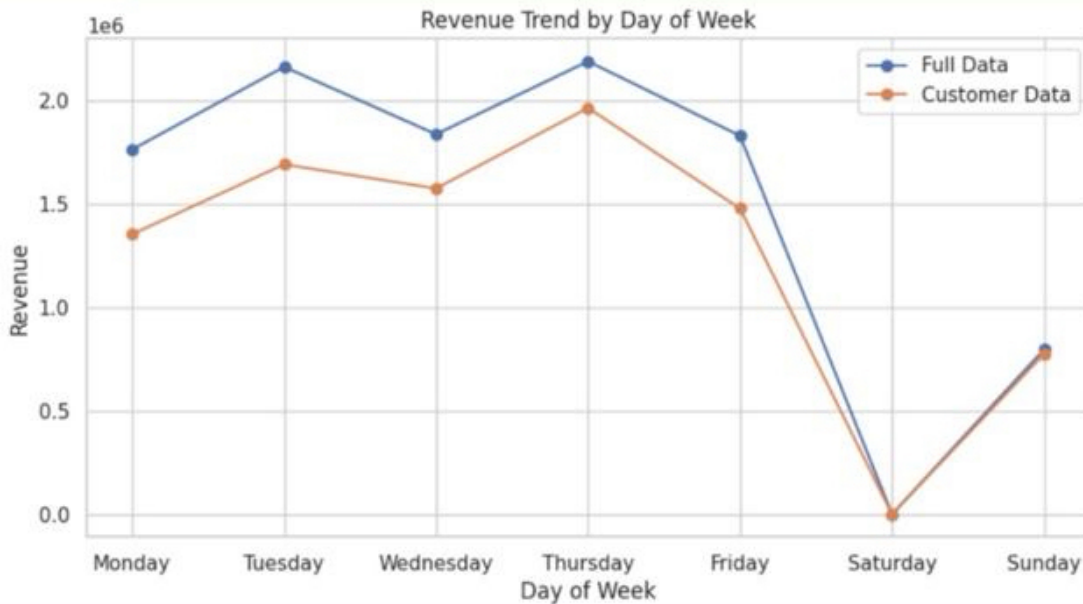
- Peak: 10:00-12:00 and 14:00-16:00
- Lowest: 0:00-6:00

Monthly Revenue Comparison

```
plt.figure(figsize=(10,5))
df_full.groupby('Month')['Revenue'].sum().plot(marker='o', label='Full Data')
df_customers.groupby('Month')['Revenue'].sum().plot(marker='o',
label='Customer Data')
plt.title("Monthly Revenue Comparison")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.legend()
plt.show()
```



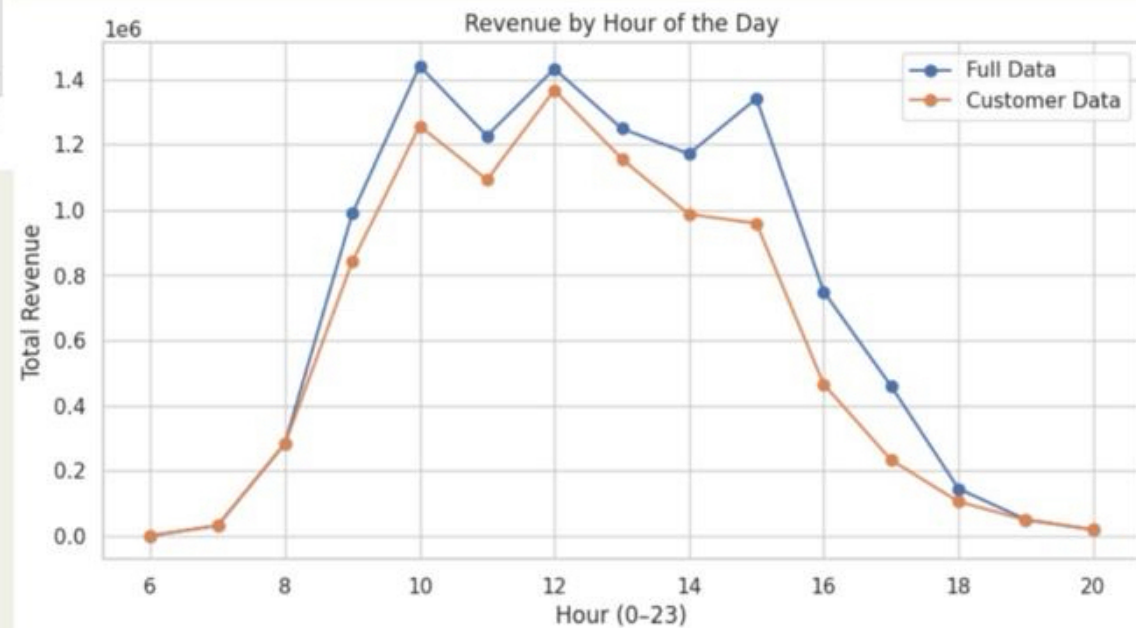
Time Series Analysis



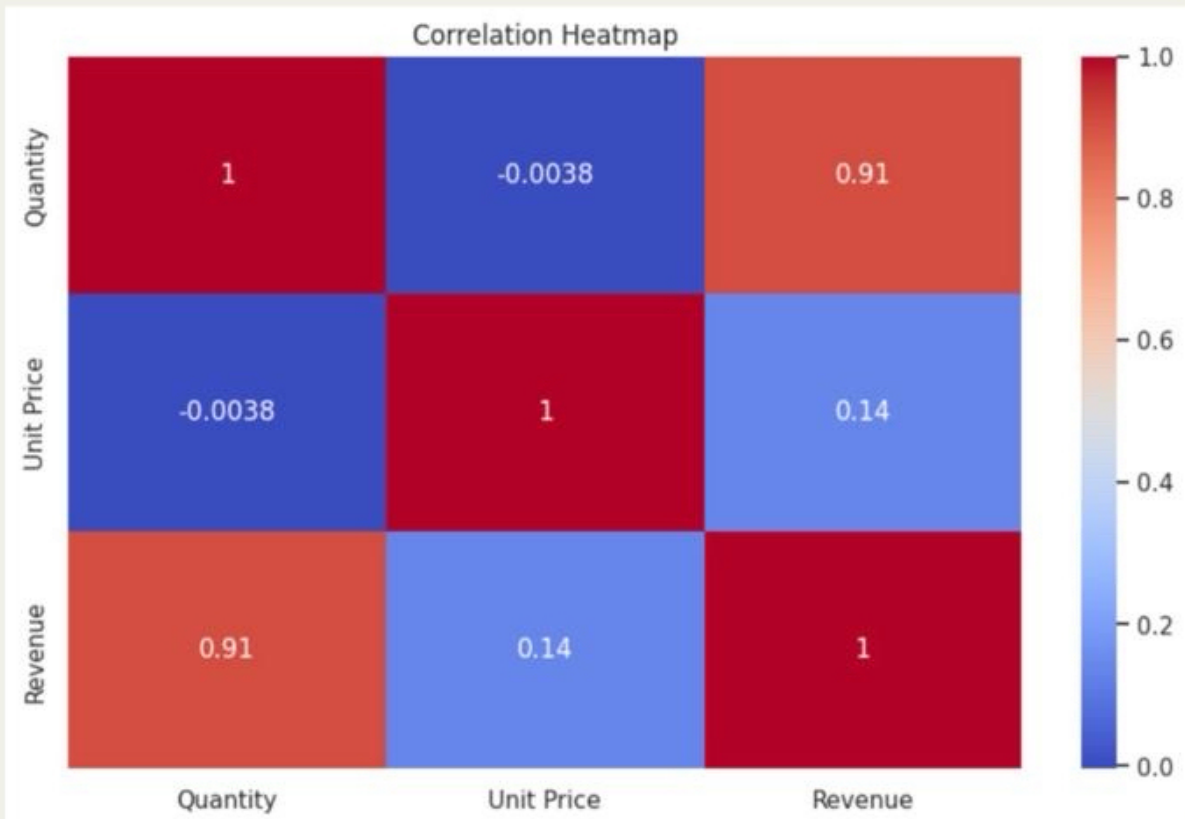
Revenue by a Day of the Week



Hourly Sales



Correlation Heatmap



```
# Heatmap (Correlation) for Quantity, Unit Price and Revenue
corr = df_full[['Quantity', 'Unit Price', 'Revenue']].corr()

sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

Product Analysis

Top 5 Products by Quantity:

- PAPER CRAFT, LITTLE BIRDIE - 80,995
- WHITE HANGING HEART T-LIGHT HOLDER - 23,333
- REGENCY CAKESTAND 3 TIER - 20,014
- JUMBO BAG RED RETROSPOT - 18,237
- SET OF 3 CAKE TINS PANTRY DESIGN - 15,876

Top 5 by Revenue (similar but some high-price items rank higher):

- REGENCY CAKESTAND 3 TIER among top

Insight:

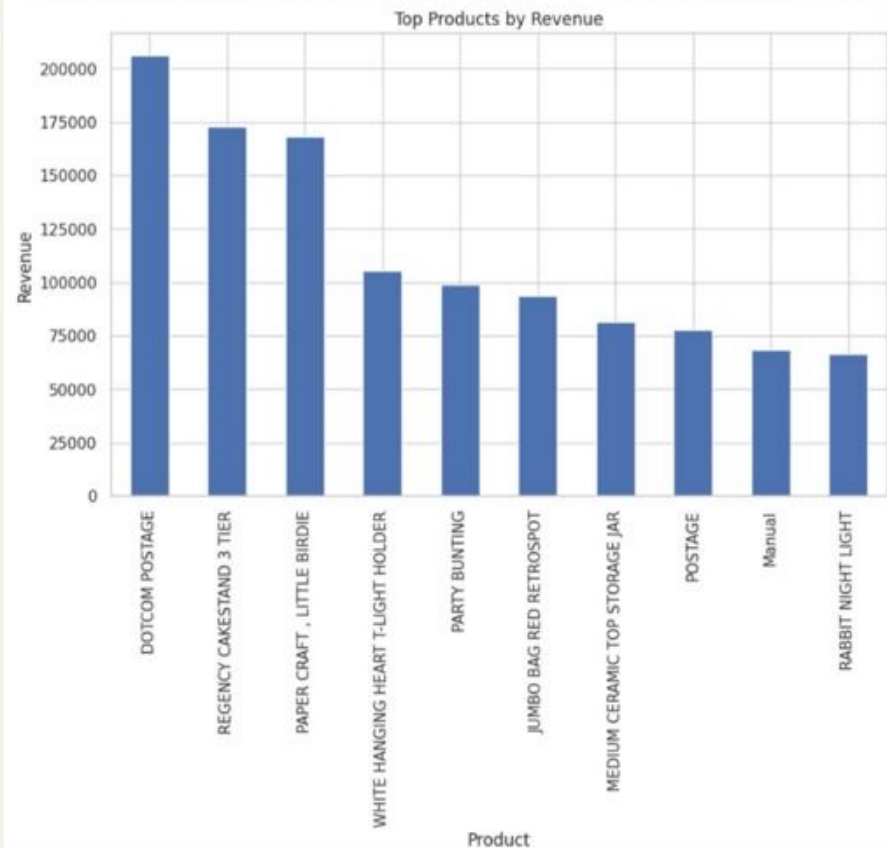
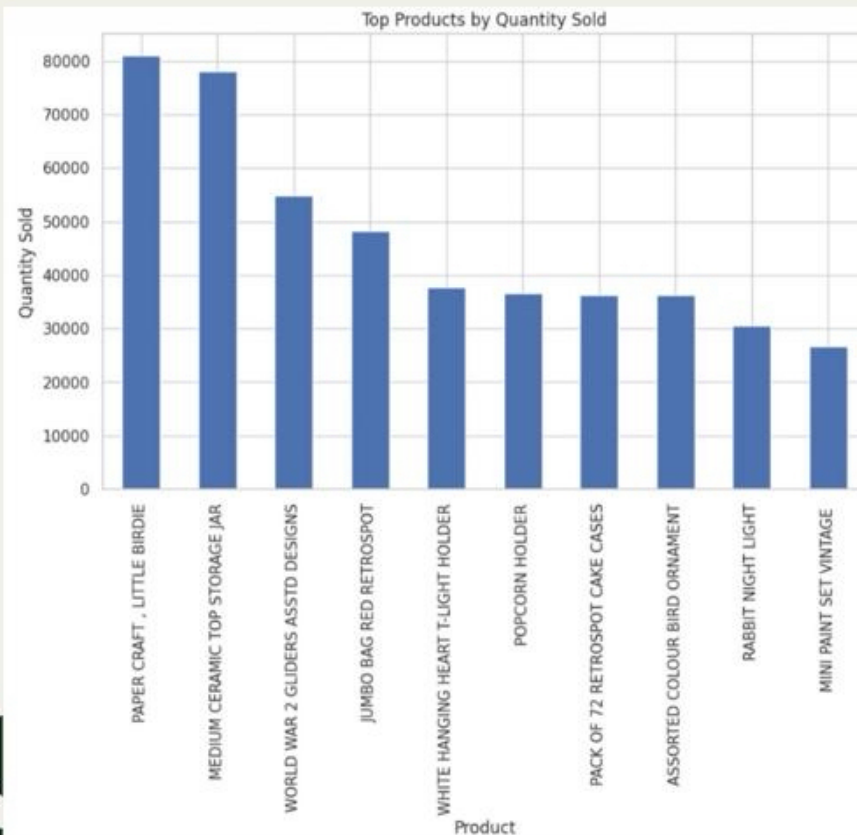
Mix of volume-driven and value-driven products.

Top 10 Products by Quantity

```
top_products = df_full.groupby('Description')
['Quantity'].sum().sort_values(ascending=False).head(10)
top_products.plot(kind='bar')
plt.title("Top Products by Quantity Sold")
plt.xlabel("Product")
plt.ylabel("Quantity Sold")
plt.xticks(rotation=90)
plt.show()
```



Product Analysis



Top 10 Products by Quantity Sold and Revenue

Country Analysis

Top 5 Countries by Revenue (Full Dataset):

- United Kingdom
- Netherlands
- EIRE (Ireland)
- Germany
- France

Top 5 Countries by Quantity (Full Dataset):

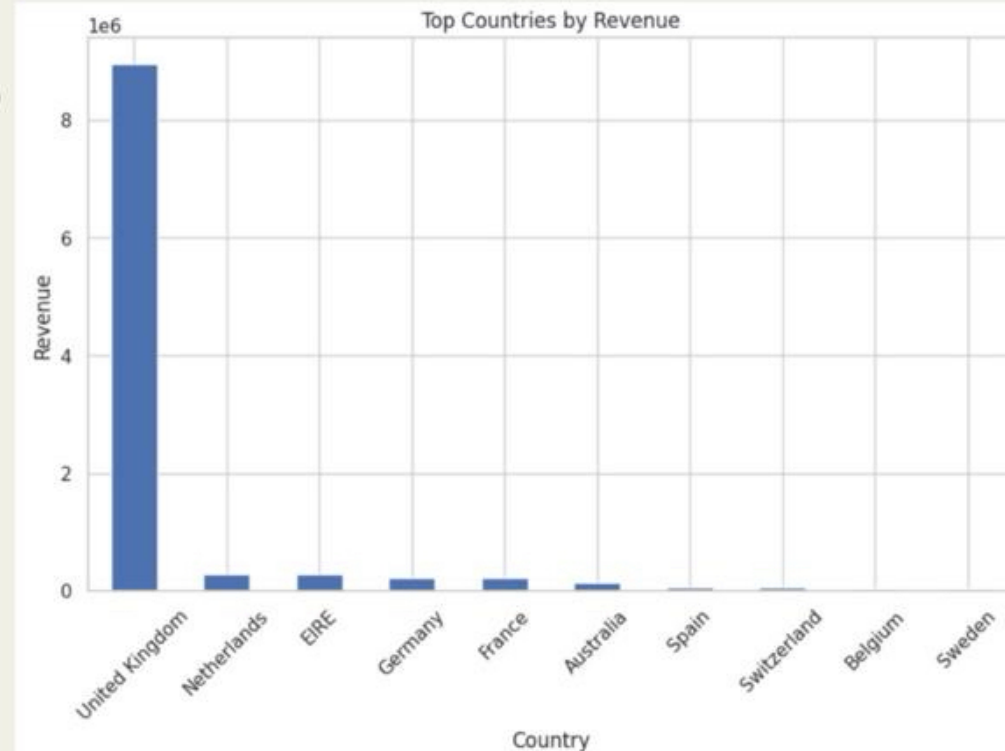
- The top 5 are the same as above.

Insights:

- UK dominates – home market strength.
- EU countries are secondary markets.
- Opportunity for targeted international expansion.

Top 10 Countries by Revenue

```
country_rev = df_full.groupby('Country')  
['Revenue'].sum().sort_values(ascending=False).head(10)  
country_rev.plot(kind='bar')  
plt.title("Top Countries by Revenue")  
plt.xlabel("Country")  
plt.ylabel("Revenue")  
plt.xticks(rotation=45)  
plt.show()
```



Customer Analysis

Top 5 Customers by Revenue:

- 1.14646 - £280,206
- 2.18102 - £259,658
- 3.17450 - £195,000
- 4.16446 - £168,470
- 5.14911 - £147,864

Top 5 Customers by Quantity:

These are different than the top 5 customers by Revenue.

Repeat vs One-time:

- Repeat customers: 2,845 (65.6%)
- One-time customers: 1,493 (34.4%)

Implications:

- High revenue concentration - key account management critical.
- Strong repeat rate, but still room to convert one-time buyers.



Customer Type (Repeat vs One-time)

Customer Analysis

Top 10 Customers by Revenue

Top 10 customers from FULL dataset (will ignore missing CustomerID automatically)

```
top_customers_full = (  
df_full[df_full['Customer ID'].notna()]  
.groupby('Customer ID')['Revenue']  
.sum()  
.sort_values(ascending=False)  
.head(10)  
)
```

Top 10 customers from CUSTOMER dataset (already cleaned)

```
top_customers_customer =(  
df_customers.groupby('Customer ID')['Revenue']  
.sum()  
.sort_values(ascending=False)  
.head(10)  
)
```

Plot side-by-side

```
import matplotlib.pyplot as plt  
fig, axes = plt.subplots(1, 2, figsize=(14,5))
```

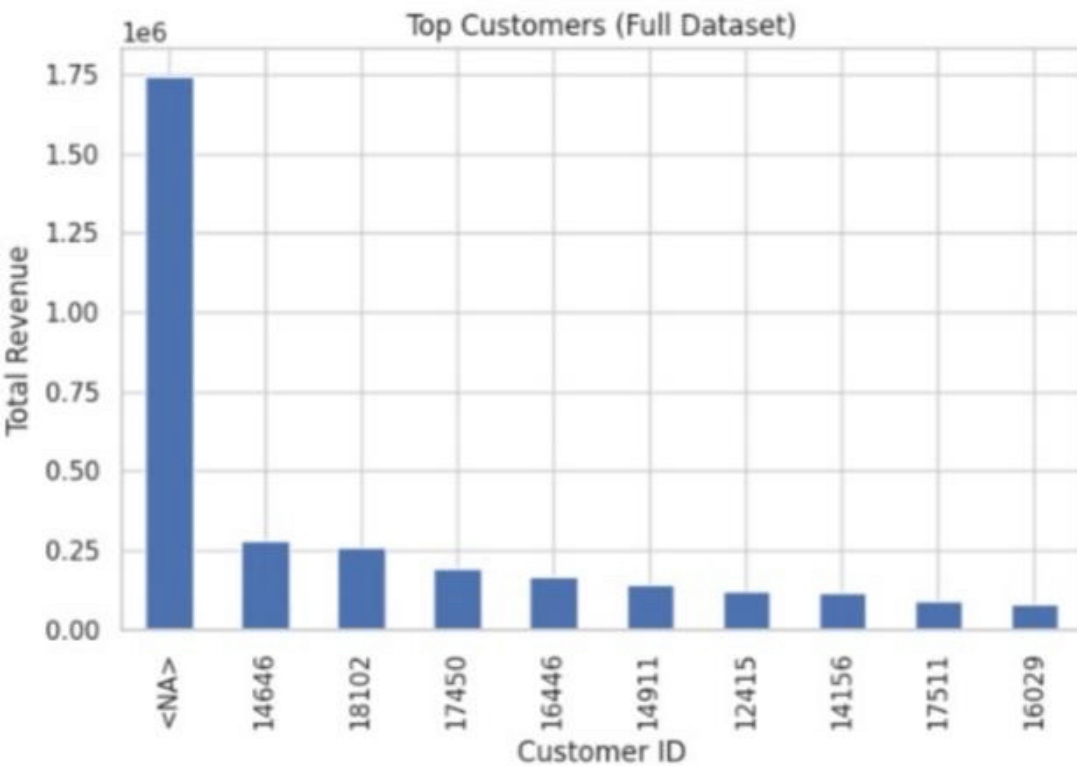
FULL DATASET

```
top_customers_full.plot(kind='bar', ax=axes[0])  
axes[0].set_title("Top Customers (Full Dataset)")  
axes[0].set_xlabel("Customer ID")  
axes[0].set_ylabel("Total Revenue")
```

CUSTOMER DATASET

```
top_customers_customer.plot(kind='bar', ax=axes[1])  
axes[1].set_title("Top Customers (Customer Dataset)")  
axes[1].set_xlabel("Customer ID")  
axes[1].set_ylabel("Total Revenue")  
plt.tight_layout()  
plt.show()
```

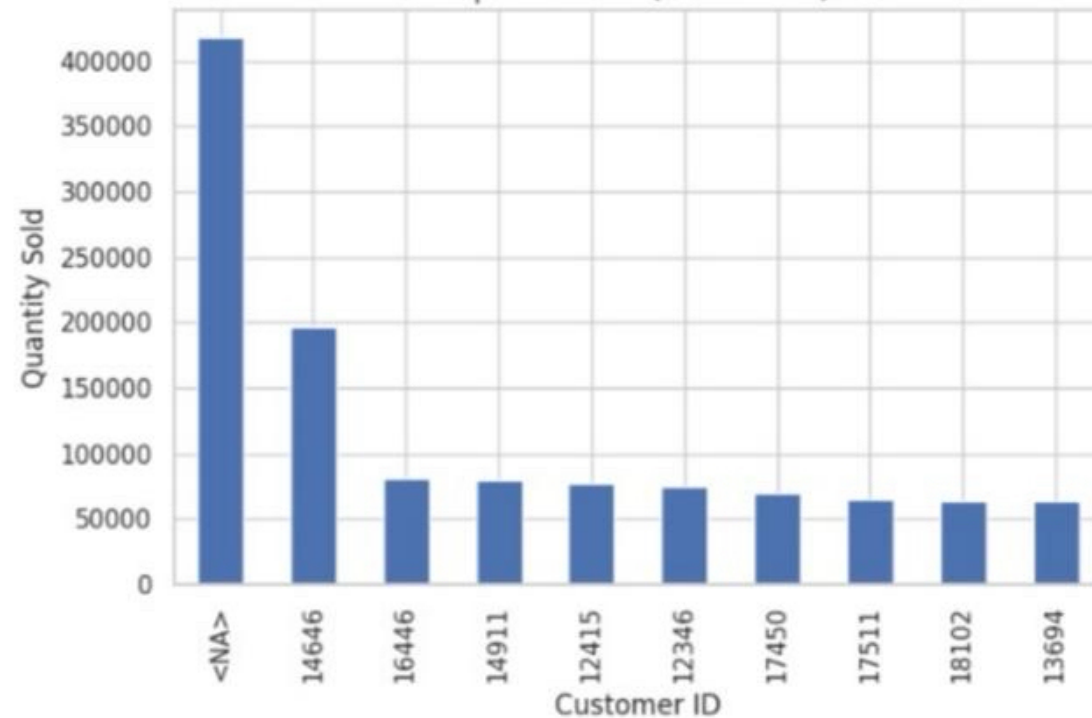

Customer Analysis



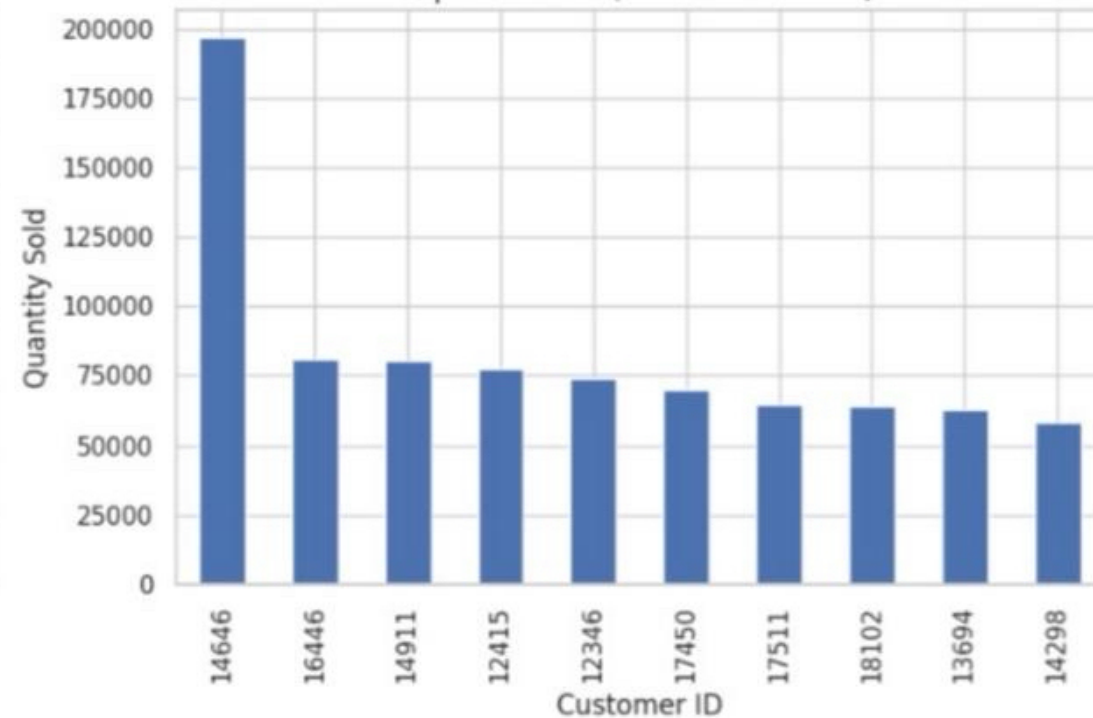
Top 10 Customers by Revenue

Customer Analysis

Top Customers (Full Dataset)



Top Customers (Customer Dataset)



Top 10 Customers by Quantity Sold

RFM Analysis

- **What is RFM?** Recency, Frequency, Monetary – a classic customer segmentation technique.
- **How we did it:**
 - Scored each customer from 1 (low) to 4 (high) on Recency, Frequency, Monetary.
 - Combined scores to create segments.
- **Segments we identified:**
 - Champions (444): Best customers – recent, frequent, high spend.
 - Loyal Customers (44x): Frequent, but may be slightly less recent.
 - Potential Loyalists (33x): Recent, decent frequency.
 - At Risk (1-2, 4): Used to buy frequently but haven't returned.
 - Lost Customers (1-2, 1-2): Inactive for a long time.
- **Insight:** 20% of customers generate 80% of revenue (Pareto principle).

RFM Analysis

```
import datetime as dt

snapshot = df_customers['Invoice Date'].max() + dt.timedelta(days=1)

rfm = df_customers.groupby('Customer ID').agg({
    'Invoice Date': lambda x: (snapshot - x.max()).days,
    'Invoice No': 'nunique',
    'Revenue': 'sum'
})

rfm.columns = ['Recency', 'Frequency', 'Monetary']

# RFM Scores
rfm['R'] = pd.qcut(rfm['Recency'], 4, labels=[4,3,2,1])
rfm['F'] = pd.qcut(rfm['Frequency'].rank(method='first'), 4, labels=[1,2,3,4])
rfm['M'] = pd.qcut(rfm['Monetary'], 4, labels=[1,2,3,4])

rfm['RFM_Score'] = rfm[['R','F','M']].astype(str).sum(axis=1)

rfm.head()
```

	Recency	Frequency	Monetary	R	F	M	RFM_Score	
Customer ID								
12346	326	1	77183.60	1	1	4	114	
12347	2	7	4310.00	4	4	4	444	
12348	75	4	1595.64	2	3	3	233	
12349	19	1	1757.55	3	1	4	314	
12350	310	1	334.40	1	1	2	112	

Cohort Analysis

Cohort Analysis: Tracks groups of customers who started in the same month over time.

Why? Measures retention, loyalty, and customer lifecycle.

Method:

- **Cohort** = First purchase month.
- **Cohort Index** = Months since first purchase.
- **Retention Rate** = % of customers who return in each subsequent month.

Insight: Retention drops sharply after first month (60-80% lost), then stabilizes. Early cohorts show better long-term retention.

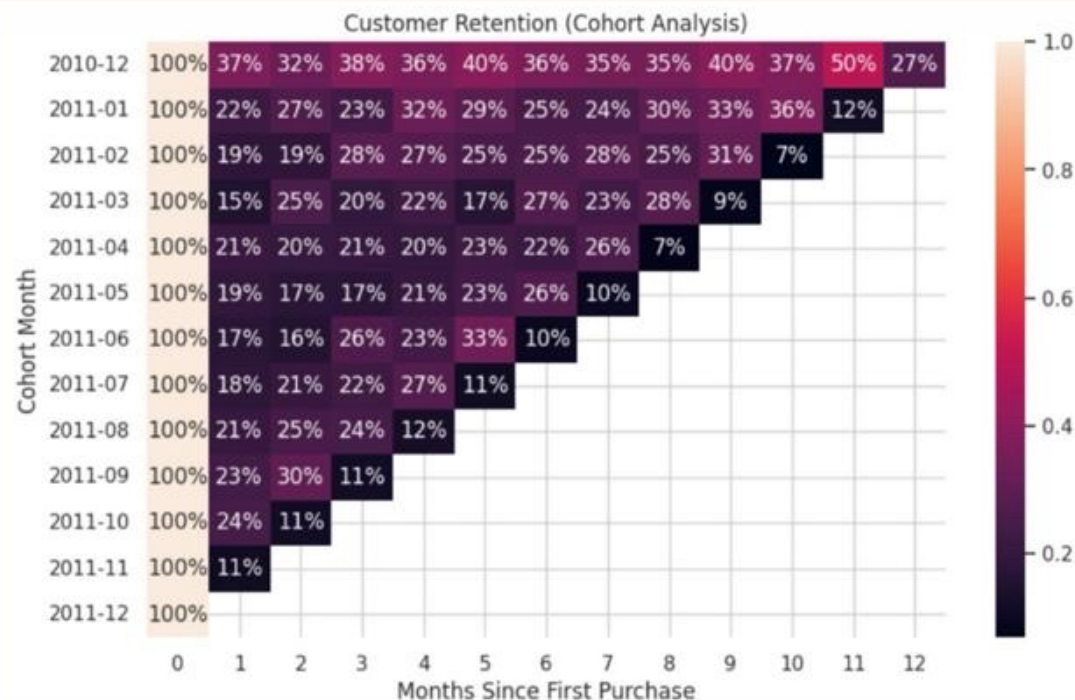


Cohort Retention Heatmap

Key Findings:

- Month 1: Retention drops to 20-40% (60-80% churn)
- Month 3: Stabilizes at 15-25%
- Month 12: Only 5-10% remain

Best Cohort: December 2010 - 27% retention at 12 months



Cohort Analysis

```
df_customers['InvoiceMonth'] = df_customers['Invoice Date'].dt.to_period('M')
df_customers['CohortMonth'] = df_customers.groupby('Customer ID')['Invoice Date'].transform('min').dt.to_period('M')

df_customers['CohortIndex'] = (df_customers['InvoiceMonth'] - df_customers['CohortMonth']).apply(lambda x: x.n)

# Cohort Table
cohort = df_customers.groupby(['CohortMonth', 'CohortIndex'])['Customer ID'].nunique().reset_index()

cohort_pivot = cohort.pivot(index='CohortMonth', columns='CohortIndex', values='Customer ID')

retention = cohort_pivot.divide(cohort_pivot.iloc[:,0], axis=0)

sns.heatmap(retention, annot=True, fmt=".0%")
plt.title("Customer Retention (Cohort Analysis)")
plt.xlabel("Months Since First Purchase")
plt.ylabel("Cohort Month")
plt.show()
```

CLV Analysis

What is CLV? Total revenue a customer will generate during their relationship with the business.

Calculation (simplified):

- Average Purchase Value × Average Purchase Frequency × Average Lifespan

Results:

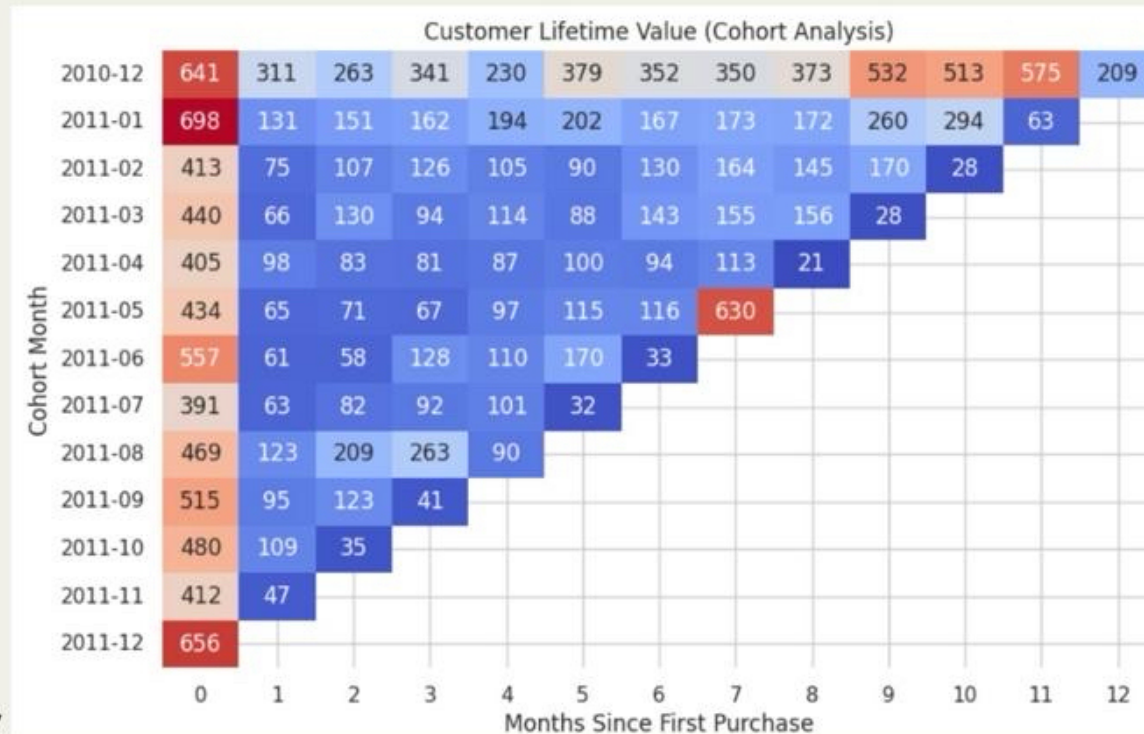
- Average CLV: £2,040
- Top 10% of customers have CLV > £10,000
- Bottom 50% have CLV < £500

Segmentation:

- Platinum (top 5%): CLV > £10,000 - VIP treatment
- Gold (next 10%): CLV £5,000-£10,000 - loyalty rewards
- Silver (20%): CLV £1,000-£5,000 - cross-sell
- Bronze (30%): CLV £500-£1,000 - engagement
- Basic (35%): CLV < £500 - cost-efficient acquisition

Insight: Allocate marketing budget based on predicted CLV.

Customer Lifetime Value



SQL Analysis and Key Queries

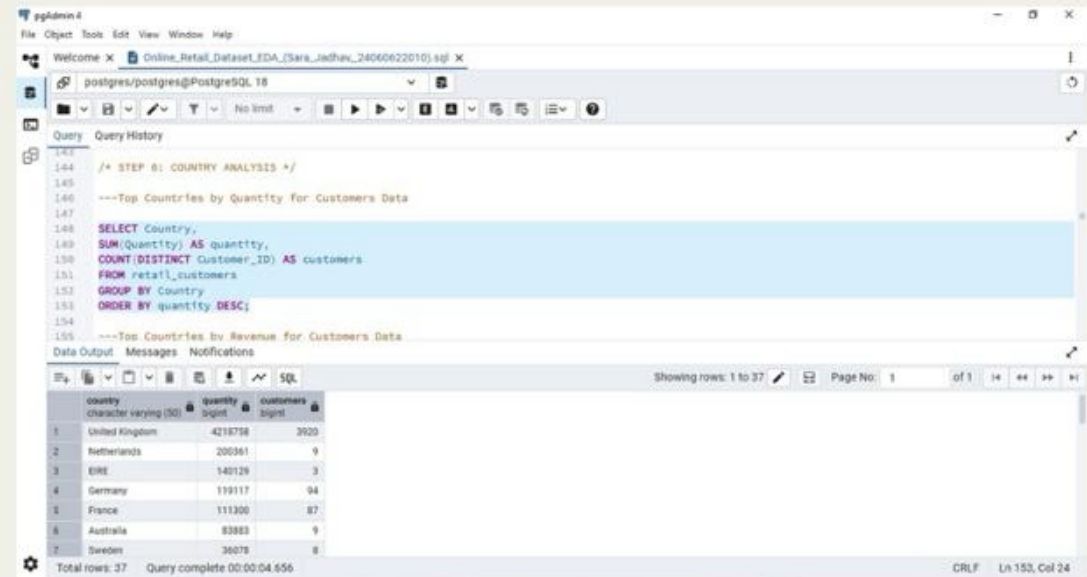
Why SQL?

- Validate Python findings.
- Perform complex analytical queries efficiently.
- Create reusable queries for business intelligence.

Key SQL Queries:

1. RFM Analysis – using NTILE and CASE statements.
2. Cohort Retention – using FIRST_VALUE window function.
3. Revenue Concentration – using RANK and FILTER.
4. Price Sensitivity – using CORR function.
5. Top Products/Countries – using GROUP BY and ORDER BY.

EDA done using SQL is given in more detail in the project report and it follows the same flow.



The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query is titled "STEP 8: COUNTRY ANALYSIS" and is a SELECT statement that calculates the sum of quantity and the count of distinct customer IDs for each country. The query is executed, and the results are displayed in a table below the editor.

```
-- STEP 8: COUNTRY ANALYSIS --  
---Top Countries by Quantity for Customers Data  
SELECT Country,  
SUM(quantity) AS quantity,  
COUNT(DISTINCT Customer_ID) AS customers  
FROM retail_customers  
GROUP BY Country  
ORDER BY quantity DESC;
```

country	quantity	customers
United Kingdom	4218758	3020
Netherlands	200361	9
IRE	140128	3
Germany	119117	94
France	111300	87
Australia	83883	9
Sweden	30078	8

Data Output										Showing rows: 1 to 10	
	customer_id	recency	frequency	monetary	r_score	f_score	m_score	rfm_score	customer_segment		
	character varying (20)	double precision	bigint	numeric	integer	integer	integer	text	text		
1	14090	324	1	76.32	5	5	5	555	Champions		
2	16756	214	1	239.40	5	5	5	555	Champions		
3	15729	212	1	85	5	5	5	555	Champions		
4	17925	372	1	244.08	5	5	5	555	Champions		
5	15923	372	1	127.08	5	5	5	555	Champions		

Limitations & Challenges



Extreme outliers

Affect averages; used medians and percentiles.



Missing Customer IDs (24.9%)

Limits customer-level analysis.



Only 12 months of data

Cannot capture full seasonality.



No external factors

Marketing campaigns, economic conditions unknown.



Log transformation required

Careful interpretation.

Lessons Learned

1 Data Quality is Critical

24.9% missing Customer IDs significantly impacts analysis depth

3 Multiple Data Views

Creating separate datasets for different analytical purposes is essential

2 Transformation Matters

Log transformations reveal patterns hidden in raw data

4 Validation is Key

SQL validation confirmed Python findings, increasing confidence



Key Takeaways

Area	Key Finding	Business Implication
Revenue	£10.6M total, £530 AOV	Strong business performance
Customers	65.6% repeat rate	Good loyalty, room for improvement
Concentration	Top 10 customers = 15-20% revenue	Key account management critical
Timing	Mid-week, business hours peak	Optimise marketing timing
Geography	UK dominates (77%)	Expand international
Retention	60-80% drop after month 1	Focus on first 30 days
CLV	£2,040 average	Target high-CLV segments

Future Recommendations



Short-term (0-3 months)

- Implement first-30-day retention program.
- Schedule marketing for mid-week and peak hours.
- Fix missing Customer ID tracking.



Medium-term (3-6 months)

- Develop loyalty program based on RFM tiers.
- Expand into Netherlands, Ireland, Germany.
- Optimize product mix (promote high-margin items).



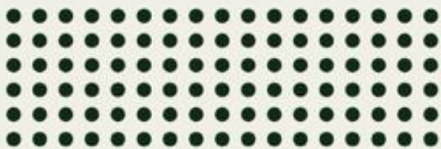
Long-term (6-12 months)

- Build predictive churn and next-purchase models.
- Automate RFM-triggered campaigns.
- Allocate marketing budget based on predicted CLV.



Conclusion

- Successfully performed comprehensive EDA, revealing customer segments, sales patterns, and retention dynamics.
- Used Python for initial analysis, SQL for validation and advanced queries.
- Derived actionable insights to improve customer retention, marketing efficiency, and revenue growth.
- Project demonstrates end-to-end data science workflow from raw data to business recommendations.



Thank You

THANK YOU FOR YOUR ATTENTION

